Projet Life Insurance

Moteur actuariel (TGF05 générationnelle) et approximation par Machine Learning

Ethan Ada & Tom Cohen

28 février 2026

Table des matières

1 Introduction **4**

2 Produit, paramètres et conventions **5**
2.1 Paramètres du contrat 5
2.2 Conventions de paiement 5
2.2.1 Contrainte contractuelle sur la durée des primes 5
2.3 Actualisation 6

3 Table de mortalité TGF05 et notion de génération **7**
3.1 Pourquoi une table générationnelle ? 7
3.2 Organisation de la table 7
3.3 Choix de génération : approche retenue 7

4 Formules actuarielles implémentées **8**
4.1 Notations et conventions 8
4.2 Construction de q_x et p_x à partir de l_x 8
4.3 Probabilité de survie ${}_tp_x$ 9
4.4 Rente temporaire différée : valeur actuelle actuarielle 9
4.5 Prime unique nette 9
4.6 Annuity-due temporaire (primes) : valeur actuelle 9
4.7 Prime annuelle nette 10
4.8 Réserve prospective V_t 10
4.8.1 Prestations futures 10
4.8.2 Primes futures 10
4.8.3 Formule finale et interprétation 10

5 Implémentation (Python) : principes **12**
5.1 Organisation du code 12
5.2 Lecture de la table et pré-traitement 12
5.2.1 Importation de TGF05 12
5.2.2 Construction de q_x et p_x 12
5.2.3 Robustesse et bornage 13
5.3 Fonctions cœur du moteur actuariel 13
5.3.1 Survie ${}_tp_x$: fonction `tpx` 13
5.3.2 Prime unique : `single_premium` 13
5.3.3 Prime annuelle : `annual_premium` 13
5.3.4 Réserve : `reserve_Vt` 13
5.4 Génération du dataset : reproductibilité 14
5.5 Vérification de cohérence (sanity checks) 14
5.5.1 Tests numériques de base 14
5.5.2 Test sur un contrat fixe 14
5.5.3 Interprétation des réserves négatives 14
5.6 Lien avec l'application Streamlit 15

6 Construction du dataset synthétique **16**
6.1 Grilles de valeurs (énoncé) . . . 16
6.2 Génération et cohérence générationnelle . . . 16
6.3 Variable temporelle t . . . 16
6.4 Variables cibles (targets) . . . 16
6.5 Contrôles de qualité . . . 17

7 Étude descriptive du dataset **18**
7.1 Structure générale du dataset . . . 18
7.2 Statistiques descriptives des variables cibles . . . 18
7.2.1 Prime unique (`single_premium`) . . . 18
7.2.2 Prime annuelle (`annual_premium`) . . . 19
7.2.3 Réserve (`reserve`) . . . 19
7.3 Analyse des paramètres explicatifs . . . 19
7.3.1 Distribution des paramètres discrets . . . 19
7.3.2 Effet qualitatif du taux i . . . 19
7.3.3 Influence de l'âge et de la génération . . . 19
7.4 Relations entre variables (analyse conceptuelle) . . . 20
7.5 Conclusion de l'étude descriptive . . . 20

8 Machine Learning : modèles et protocole **21**
8.1 Objectif ML . . . 21
8.2 Features et targets . . . 21
8.3 Préparation des données . . . 22
8.3.1 Séparation Train/Test . . . 22
8.3.2 Pourquoi un simple split ? . . . 22
8.4 Modèles testés . . . 22
8.4.1 Baseline linéaire : Ridge Regression . . . 22
8.4.2 Modèle non-linéaire : HistGradientBoostingRegressor (HGB) . . . 23
8.5 Métriques d'évaluation . . . 23
8.5.1 R^2 (coefficient de détermination) . . . 23
8.5.2 MAE (Mean Absolute Error) . . . 23
8.5.3 RMSE (Root Mean Squared Error) . . . 23
8.6 Learning Curves . . . 24
8.7 Feature Importance . . . 24
8.8 Synthèse méthodologique . . . 24

9 Résultats ML **25**
9.1 Scores obtenus . . . 25
9.2 Commentaires . . . 25

10 Learning curves : évolution des performances en fonction de N **26**
10.1 Résultats attendus . . . 26
10.2 Interprétation des learning curves . . . 26

11 Feature importance **28**
11.1 Intuition . . . 28

12 Application Streamlit **29**
12.1 Démarche de développement . . . 29
12.2 Architecture de l'application . . . 29
12.3 Choix de conception . . . 30
12.4 Apport pédagogique . . . 30

13 Conclusion **31**

Chapitre 1

Introduction

L'objectif du projet est double :

— Implémenter un **moteur actuariel** pour un produit d'assurance vie basé sur une table de mortalité **générationnelle** (TGF05).

— Construire un **dataset synthétique** à partir de ce moteur actuariel puis entraîner des modèles de **Machine Learning** afin d'approximer les quantités actuarielles (prime unique, prime annuelle et réserve).

La motivation principale est de montrer qu'une formule actuarielle déterministe (combinant survie, actualisation et produits de probabilités) peut être apprise par un modèle statistique, et d'étudier comment la performance varie en fonction de la taille du dataset et du choix du modèle.

Dans la suite, on décrit :

— le produit et les conventions,

— la table de mortalité TGF05 et la notion de génération,

— les formules actuarielles utilisées,

— la génération du dataset synthétique,

— l'étude descriptive du dataset,

— l'entraînement ML (baseline linéaire vs modèle non-linéaire),

— les métriques, learning curves et feature importance,

— enfin une application Streamlit pour la démonstration (moteur actuariat vs moteur ML).

Utilisation d'outils d'IA

Dans le cadre de ce projet, des outils d'intelligence artificielle générative ont été utilisés comme assistance ponctuelle à la rédaction et au débogage.

Ces outils ont principalement servi à :

— reformuler certaines parties rédactionnelles ;

— vérifier la cohérence syntaxique de portions de code Python ;

— suggérer des pistes d'amélioration de structuration du rapport.

En revanche, les choix actuariels (conventions, formules, indices), la modélisation des produits, la construction du moteur de tarification, la définition des contraintes contractuelles ainsi que l'analyse méthodologique des performances des modèles de Machine Learning ont été réalisés de manière autonome.

L'outil n'a donc pas été utilisé comme substitut au raisonnement actuariel, mais comme support technique.

Chapitre 2

Produit, paramètres et conventions

2.1 Paramètres du contrat

Le contrat est paramétré par :

— x : âge à la souscription,

— m : durée de paiement des primes (*annuity-due*, paiement en début d'année),

— m' : différé avant le début des prestations,

— n : durée maximale de service de la rente (*annuity-immediate*, paiement fin d'année),

— i : taux technique (constant),

— A : montant de la rente payée à chaque échéance.

2.2 Conventions de paiement

— Les **primes** sont payées **en début d'année** pendant m années (annuity-due).

— Les **prestations** (rente) sont payées **en fin d'année** après un différé m' et au plus pendant n années (annuity-immediate).

— La prestation à l'échéance $k+1$ n'est payée que si l'assuré est vivant à cette date.

2.2.1 Contrainte contractuelle sur la durée des primes

Théoriquement, la seule contrainte nécessaire serait :

$$m \leq m' + n,$$

qui garantit que la durée de paiement des primes ne dépasse pas la durée totale du contrat.

Cependant, dans ce projet, nous retenons une contrainte plus restrictive :

$$m \leq m'.$$

Cette hypothèse signifie que l'intégralité des primes est versée avant le début du service de la rente.

Ce choix est cohérent d'un point de vue produit et permet d'éviter des configurations économiquement atypiques (paiement de primes pendant la phase de service).

Cette contrainte est implémentée : — dans la génération du dataset, — dans le moteur actuariel, — et contrôlée dans l'application Streamlit.

2.3 Actualisation

On utilise le facteur d'actualisation :

$$v(i) = \frac{1}{1+i}.$$

Chapitre 3

Table de mortalité TGF05 et notion de génération

3.1 Pourquoi une table générationnelle ?

Contrairement à une table « statique » où q_x dépend uniquement de l'âge, la table TGF05 est **générationnelle** : les probabilités de décès et les effectifs survivants dépendent de :

— l'âge x,

— **l'année de naissance** (génération).

Intuitivement, deux individus du même âge mais nés à des périodes différentes n'ont pas la même mortalité : les générations récentes bénéficient généralement d'une mortalité plus faible (amélioration de l'espérance de vie).

3.2 Organisation de la table

Dans le fichier TGF05 :

— les lignes correspondent aux âges,

— les colonnes correspondent aux **années de naissance** (générations),

— les valeurs sont l_x (effectif survivant à l'âge x).

3.3 Choix de génération : approche retenue

L'énoncé impose la variable **generation** comme **année de naissance**. Dans le moteur, on utilise une génération **cohérente** :

$$\text{generation} = \text{issue_year} - x,$$

où `issue_year` est l'année de souscription (paramètre). Cela garantit que la génération est bien une année de naissance et permet de garder la dimension générationnelle dans le modèle.

Chapitre 4

Formules actuarielles implémentées

4.1 Notations et conventions

On note :

— x : âge à la souscription,

— gen : génération (année de naissance),

— i : taux technique constant, et $v = \dfrac{1}{1+i}$ le facteur d'actualisation annuel,

— $l_x(\text{gen})$: effectif survivant à l'âge x pour la génération considérée,

— $q_x(\text{gen})$: probabilité de décès entre x et $x+1$,

— $p_x(\text{gen}) = 1 - q_x(\text{gen})$: probabilité de survie sur un an.

Dans ce projet, la table TGF05 est **générationnelle** : pour un même âge x, la mortalité dépend de la génération. Dans l'implémentation, la génération est déterminée de façon cohérente via :

$$\text{gen} = \text{issue_year} - x,$$

où issue_year est l'année de souscription (paramètre contrôlé dans le dataset et dans l'application).

Les conventions de paiement sont :

— **Primes** payées en **début d'année** pendant m ans (annuity-due),

— **Prestations** payées en **fin d'année** après un différé m' et au plus pendant n ans (annuity-immediate).

4.2 Construction de q_x et p_x à partir de l_x

La table fournit $l_x(\text{gen})$. On en déduit :

$$d_x(\text{gen}) = l_x(\text{gen}) - l_{x+1}(\text{gen}), \qquad q_x(\text{gen}) = \frac{d_x(\text{gen})}{l_x(\text{gen})} = \frac{l_x(\text{gen}) - l_{x+1}(\text{gen})}{l_x(\text{gen})},$$

puis :

$$p_x(\text{gen}) = 1 - q_x(\text{gen}).$$

Convention au dernier âge. Dans le code, au dernier âge disponible dans la table, on impose une convention simple :

$$q_\omega(\text{gen}) = 1 \quad \text{si } l_\omega(\text{gen}) > 0,$$

afin d'éviter des divisions par zéro et d'assurer une terminaison cohérente.

4.3 Probabilité de survie ${}_tp_x$

La probabilité de survie pendant t années à partir de l'âge x (pour la génération gen) est :

$$ {}_tp_x(\text{gen}) = \prod_{k=0}^{t-1} p_{x+k}(\text{gen}), \qquad {}_0p_x(\text{gen}) = 1. $$

Cette quantité est centrale : elle permet de pondérer chaque flux (prime ou prestation) par la probabilité que l'assuré soit vivant au moment du paiement. Dans le code, cette probabilité est calculée explicitement via une boucle produit sur les p_{x+k}.

4.4 Rente temporaire différée : valeur actuelle actuarielle

On considère une rente de montant A payée **en fin d'année** (*immediate*) après un différé m' et au maximum pendant n années.

Le flux de rente payé à l'échéance $k+1$ (avec $k \geq m'$) n'a lieu que si l'assuré est vivant à cette date. La valeur actuelle actuarielle de la rente temporaire différée est donc :

$$ {}_{m'}a_{x:\overline{n}}(\text{gen}) = \sum_{k=m'}^{m'+n-1} v^{k+1}\, {}_kp_x(\text{gen}). $$

Interprétation. Le terme v^{k+1} actualise un paiement à la fin de l'année $k+1$. Le terme ${}_kp_x$ impose la condition de survie jusqu'au temps k.

Remarque sur l'indexation.

Une écriture équivalente peut être obtenue en factorisant le différé :

$$ {}_{m'}a_{x:\overline{n}|} = v^{m'}\, {}_{m'}p_x \sum_{k=1}^{n} v^k\, {}_kp_{x+m'}. $$

Cette forme met explicitement en évidence :

— la survie jusqu'au début du service, — puis la survie pendant la phase de paiement.

Elle permet également de vérifier la cohérence des indices avec l'énoncé fourni.

4.5 Prime unique nette

La prime unique nette U (payée à $t=0$) est égale à la valeur actuelle actuarielle des prestations :

$$ U = A \cdot {}_{m'}a_{x:\overline{n}}(\text{gen}) = A \sum_{k=m'}^{m'+n-1} v^{k+1}\, {}_kp_x(\text{gen}). $$

Remarque. Comme A est multiplicatif, la prime unique est linéaire en A. En revanche, elle est non-linéaire en i (via v^{k+1}) et en (x, gen) (via les produits de survie).

4.6 Annuity-due temporaire (primes) : valeur actuelle

Les primes annuelles nivelées P sont payées en **début d'année** pendant m ans, uniquement si l'assuré est vivant au moment du paiement.

Le paiement à l'instant j (début de l'année j) est actualisé par v^j et pondéré par la survie ${}_jp_x$. La valeur actuelle d'une annuité-due temporaire est donc :

$$ \ddot{a}_{x:\overline{m}}(\text{gen}) = \sum_{j=0}^{m-1} v^j\, {}_jp_x(\text{gen}). $$

4.7 Prime annuelle nette

Sous l'hypothèse d'équivalence actuarielle (*PV primes = PV prestations*), on obtient :

$$P \cdot \ddot{a}_{x:\overline{m}}(\text{gen}) = U \quad \Longrightarrow \quad P = \frac{U}{\ddot{a}_{x:\overline{m}}(\text{gen})}.$$

Pourquoi c'est plus difficile pour le ML ? Même si U est déjà non-linéaire, P ajoute une **division** par $\ddot{a}_{x:\overline{m}}$, ce qui rend la relation encore plus difficile à approximer avec un modèle linéaire (ce que l'on observe dans les résultats).

4.8 Réserve prospective V_t

La réserve prospective au temps t est définie comme la différence entre :

— la valeur actuelle des **prestations futures** conditionnellement à la survie à t,

— la valeur actuelle des **primes futures** conditionnellement à la survie à t.

On écrit :

$$V_t = PV_B(t) - PV_P(t).$$

4.8.1 Prestations futures

À l'instant t, l'assuré a l'âge $x + t$. Les prestations futures correspondent aux paiements en fin d'année $k + 1$ pour $k \in [m', m' + n - 1]$ et $k \geq t$. La valeur actuelle des prestations depuis t s'écrit :

$$PV_B(t) = A \sum_{k=\max(m',t)}^{m'+n-1} v^{(k+1)-t} \, {}_{k-t}p_{x+t}(\text{gen}).$$

Lecture du terme.

— $v^{(k+1)-t}$: actualisation d'un paiement à l'instant $k + 1$ vu depuis t,

— ${}_{k-t}p_{x+t}$: survie entre t et k (conditionnellement à la survie à t).

4.8.2 Primes futures

Les primes sont payées en début d'année jusqu'à $m - 1$. Si $t < m$, il reste des paiements aux instants $j = t, t + 1, \ldots, m - 1$. La valeur actuelle des primes depuis t est :

$$PV_P(t) = P \sum_{j=t}^{m-1} v^{j-t} \, {}_{j-t}p_{x+t}(\text{gen}).$$

Si $t \geq m$, il n'y a plus de primes :

$$PV_P(t) = 0.$$

4.8.3 Formule finale et interprétation

On obtient finalement :

$$V_t = A \sum_{k=\max(m',t)}^{m'+n-1} v^{(k+1)-t} \, {}_{k-t}p_{x+t}(\text{gen}) \; - \; P \sum_{j=t}^{m-1} v^{j-t} \, {}_{j-t}p_{x+t}(\text{gen}).$$

Pourquoi la réserve est la target la plus difficile ? La réserve dépend fortement du temps t et présente un comportement par morceaux :

- **avant** le début de service ($t < m'$) : les prestations n'ont pas encore commencé mais les primes peuvent encore être payées,
- **pendant** la période de service ($m' \leq t \leq m' + n$) : les prestations sont actives,
- **après** m : il n'y a plus de primes.

Cette structure « changement de régime » explique les performances ML plus faibles sur `reserve` par rapport à `single_premium`.

On peut également formaliser la réserve sous une écriture compacte intégrant une indicatrice :

$$V_t = \mathrm{VA}_t(\text{prestations futures}) - \mathbf{1}_{\{t<m\}}\mathrm{VA}_t(\text{primes futures}).$$

L'introduction de l'indicatrice $\mathbf{1}_{\{t<m\}}$ rend explicite le changement de régime intervenant à l'instant $t = m$, à partir duquel aucune prime n'est due.

Chapitre 5

Implémentation (Python) : principes

5.1 Organisation du code

L'implémentation a été structurée de manière modulaire afin de séparer clairement :

— le **moteur actuariel** (lecture de table, calculs U, P, V_t),

— la **génération du dataset** (tirage aléatoire des paramètres et calcul des targets),

— la partie **Machine Learning** (entraînement, métriques, learning curves, feature importance),

— et enfin la **démonstration applicative** via une application Streamlit (pages Actuariat / ML / Performance).

Ce choix est important pour la reproductibilité : il devient possible de régénérer un dataset ou de ré-entraîner les modèles sans modifier le moteur actuariel, et inversement.

5.2 Lecture de la table et pré-traitement

5.2.1 Importation de TGF05

La table TGF05 est importée depuis un fichier `.xls` via `pandas.read_excel`. Le format du fichier impose une étape de mise en forme :

— les **lignes** correspondent aux âges,

— les **colonnes** correspondent aux générations (années de naissance),

— les cellules contiennent $l_x(\text{gen})$.

Après lecture, on construit un `DataFrame` propre :

— index = âge (entier),

— colonnes = générations (entier),

— valeurs = l_x converties en numérique (les valeurs non convertibles sont mises à `NaN`).

5.2.2 Construction de q_x et p_x

À partir de l_x, on dérive :

$$q_x(\text{gen}) = \frac{l_x(\text{gen}) - l_{x+1}(\text{gen})}{l_x(\text{gen})}, \qquad p_x(\text{gen}) = 1 - q_x(\text{gen}).$$

Dans le code, cette étape est réalisée de façon vectorisée via un décalage (`shift(-1)`) sur les âges.

Gestion du dernier âge. La table étant finie, l_{x+1} n'est pas toujours disponible au dernier âge. Une convention simple est utilisée :

$$q_\omega(\text{gen}) = 1 \quad \text{si } l_\omega(\text{gen}) > 0,$$

ce qui évite les erreurs numériques et assure une terminaison cohérente.

5.2.3 Robustesse et bornage

Afin d'éviter des instabilités numériques (valeurs aberrantes dues à des divisions par des effectifs très faibles), on borne les probabilités :

$$q_x \in [0,1], \qquad p_x = 1 - q_x.$$

De plus, si une combinaison `(âge, génération)` est hors domaine de la table, la survie est fixée à 0 par défaut (ce qui revient à dire que le calcul n'est pas valide pour cette combinaison).

5.3 Fonctions cœur du moteur actuariel

Le moteur actuariel est implémenté sous forme de fonctions Python simples (principe : une fonction = une formule actuarielle), afin de garder une correspondance claire entre le code et les équations du rapport.

5.3.1 Survie ${}_tp_x$: fonction `tpx`

La fonction `tpx(age, gen, t)` calcule la probabilité de survie :

$${}_tp_x(\text{gen}) = \prod_{k=0}^{t-1} p_{x+k}(\text{gen}).$$

Dans l'implémentation, ce produit est calculé par une boucle, car cela rend le code lisible et permet de gérer facilement les sorties de domaine (âges non présents dans la table).

5.3.2 Prime unique : `single_premium`

La prime unique est calculée via :

$$U = A \sum_{k=m'}^{m'+n-1} v^{k+1}\, {}_kp_x(\text{gen}).$$

Le code respecte directement cette structure (somme sur k, actualisation, survie, puis multiplication par A).

5.3.3 Prime annuelle : `annual_premium`

La prime annuelle repose sur l'équivalence actuarielle :

$$P = \frac{U}{\ddot{a}_{x:\overline{m}}(\text{gen})}, \qquad \ddot{a}_{x:\overline{m}}(\text{gen}) = \sum_{j=0}^{m-1} v^j\, {}_jp_x(\text{gen}).$$

Le dénominateur (annuité-due) est calculé par une fonction dédiée, ce qui évite les duplications de code et facilite la relecture.

5.3.4 Réserve : `reserve_Vt`

La réserve est calculée par définition prospective :

$$V_t = PV_B(t) - PV_P(t),$$

avec :

$$PV_B(t) = A \sum_{k=\max(m',t)}^{m'+n-1} v^{(k+1)-t}\, {}_{k-t}p_{x+t}(\text{gen}),$$

et si $t < m$:

$$PV_P(t) = P \sum_{j=t}^{m-1} v^{j-t} \, {}_{j-t}p_{x+t}(\text{gen}).$$

Dans le code, on distingue clairement :

— la partie **prestations futures** (somme sur k),

— la partie **primes futures** (somme sur j conditionnelle à $t < m$).

Point important. La réserve dépend fortement de t et change de régime avant/après m' et avant/après m, ce qui explique la difficulté ML plus élevée pour la target `reserve`.

5.4 Génération du dataset : reproductibilité

Le dataset est généré par simulation à partir des grilles imposées dans l'énoncé. Un **seed** est fixé afin que :

— le dataset soit reproductible,

— les métriques ML soient comparables d'une exécution à l'autre,

— l'application Streamlit donne des résultats cohérents (pas de dataset qui change à chaque lancement).

Le dataset est ensuite **sauvegardé** (CSV) et rechargé lorsque nécessaire, ce qui évite de recalculer systématiquement les N lignes.

5.5 Vérification de cohérence (sanity checks)

Avant de passer à l'entraînement ML, plusieurs vérifications sont appliquées afin de s'assurer que le moteur actuariel et la génération du dataset sont cohérents.

5.5.1 Tests numériques de base

On vérifie notamment :

— **absence de primes négatives** : $U \geq 0$ et $P \geq 0$ sur le dataset,

— **bornes et ordre de grandeur** : min/max des primes et réserves plausibles,

— **valeurs manquantes** : suppression des lignes contenant `NaN` ou `Inf` après calcul.

5.5.2 Test sur un contrat fixe

Un test sur un contrat donné (valeurs fixées de $x, m, m', n, i, A, \text{gen}$) permet de :

— comparer rapidement le résultat à un calcul Excel (si disponible),

— tracer la courbe V_t pour vérifier qualitativement la dynamique (avant/après différé, avant/après fin des primes),

— détecter immédiatement un problème d'indexation âge/génération dans la table.

5.5.3 Interprétation des réserves négatives

Dans certains cas, des réserves négatives peuvent apparaître, notamment lorsque :

— t est très petit et que la valeur actuelle des primes futures dépasse celle des prestations futures,

— la convention de réserve (avant/juste après paiement de prime) influe sur le signe à court terme.

Ces cas ne sont pas nécessairement des erreurs : ils doivent être expliqués comme un effet des conventions et de la définition prospective retenue.

5.6 Lien avec l'application Streamlit

L'implémentation Python a été directement réutilisée dans une application Streamlit structurée en plusieurs pages :

— **Moteur Actuariat** : calculs exacts et courbes de réserve,

— **Moteur ML** : prédictions et écarts vs actuariel,

— **Performance & Training** : génération/sauvegarde dataset, entraînement, métriques, learning curves, feature importance.

Cette organisation permet une démonstration claire : le moteur actuariel joue le rôle de « vérité terrain » et le moteur ML montre la capacité d'approximation et les limites sur les quantités les plus complexes (notamment la réserve).

Chapitre 6

Construction du dataset synthétique

6.1 Grilles de valeurs (énoncé)

Le dataset est construit en tirant aléatoirement les paramètres dans les ensembles discrets :

- $x \in \{20, 30, 40, 50, 60\}$,
- $m \in \{1, 5, 10, 20, 30, 40\}$,
- $m' \in \{0, 1, 5, 10, 20, 30, 40\}$,
- $n \in \{1, 5, 10, 20, 30, 40, 50, 60\}$,
- $i \in \{0, 0.5\%, 1\%, 1.5\%, 2\%, 2.5\%\}$,
- $A \in \{50, 100, 200, 400, 800, 1000, 2000\}$.

La taille du dataset est $N \in [1, 1000]$ (consigne).

6.2 Génération et cohérence générationnelle

On introduit un `issue_year` (année de souscription) et on impose :

$$\text{generation} = \text{issue_year} - x.$$

On choisit `issue_year` dans une plage raisonnable (ex. 1990–2025) en veillant à ce que la génération reste dans les bornes de la table TGF05.

6.3 Variable temporelle t

On tire t uniformément dans :

$$t \in \{0, 1, \ldots, m' + n\},$$

afin de couvrir toute la zone utile pour la réserve.

6.4 Variables cibles (targets)

Pour chaque ligne, le moteur actuariel produit :

- `single_premium` : U,
- `annual_premium` : P,
- `reserve` : V_t.

6.5 Contrôles de qualité

Des vérifications automatiques sont effectuées :

— absence de primes négatives,

— statistiques min/max raisonnables,

— absence de valeurs manquantes après génération.

Chapitre 7

Étude descriptive du dataset

L'étude descriptive du dataset a pour objectif de vérifier la cohérence statistique des données générées à partir du moteur actuariel et d'analyser qualitativement la structure des variables avant l'étape de Machine Learning.

Le dataset est composé de N observations (avec $1 \leq N \leq 1000$ conformément à l'énoncé), chacune correspondant à un contrat caractérisé par les paramètres :

$$(x, m, m', n, i, A, \text{generation}, t)$$

et les variables cibles :

`single_premium`, `annual_premium`, `reserve`.

7.1 Structure générale du dataset

Chaque ligne du dataset correspond à une combinaison aléatoire de paramètres discrets issus des grilles imposées par l'énoncé.

On vérifie en premier lieu :

— la taille effective du dataset (après suppression éventuelle de valeurs manquantes),
— l'absence de valeurs infinies ou non numériques,
— la cohérence du type des variables (entiers pour x, m, m', n, t, réels pour i, A et les targets).

Cette étape garantit que le dataset est exploitable par les modèles statistiques.

7.2 Statistiques descriptives des variables cibles

Pour chaque variable cible, on calcule :

— minimum,
— maximum,
— moyenne,
— écart-type,
— quantiles (25%, 50%, 75%).

7.2.1 Prime unique (`single_premium`)

La prime unique présente les caractéristiques suivantes :

— elle est toujours positive (absence de primes négatives),
— elle varie fortement selon le montant A et le taux i,
— elle augmente avec la durée n et diminue lorsque le différé m' augmente.

L'amplitude observée (valeurs minimales faibles et valeurs maximales élevées) reflète l'hétérogénéité volontaire introduite dans les paramètres.

7.2.2 Prime annuelle (`annual_premium`)

La prime annuelle :

— est également positive sur l'ensemble du dataset,
— dépend fortement de la durée de paiement m,
— peut être très élevée lorsque m est faible (car la prime unique est amortie sur peu d'années).

La dispersion observée est cohérente avec la structure :

$$P = \frac{U}{\ddot{a}_{x:\overline{m}}}.$$

7.2.3 Réserve (`reserve`)

La réserve présente une structure plus complexe :

— elle peut être positive ou négative selon la valeur de t,
— elle dépend fortement du régime (avant ou après début de service),
— elle présente une variabilité importante en fonction de la combinaison (m, m', n, t).

La présence de réserves négatives n'est pas nécessairement incohérente : elle peut s'expliquer par la définition prospective retenue et par le fait que, à certains instants précoces, la valeur actuelle des primes futures excède celle des prestations futures.

7.3 Analyse des paramètres explicatifs

7.3.1 Distribution des paramètres discrets

On vérifie que les valeurs des paramètres x, m, m', n, i, A sont réparties conformément aux grilles définies dans l'énoncé.

Chaque valeur possible apparaît un nombre comparable de fois (tirage uniforme), ce qui garantit :

— l'absence de biais systématique en faveur d'un âge ou d'un taux particulier,
— une couverture large de l'espace des paramètres,
— un apprentissage ML basé sur des configurations variées.

7.3.2 Effet qualitatif du taux i

L'analyse des moyennes conditionnelles montre que :

— lorsque le taux i augmente, la valeur actuelle des prestations diminue,
— par conséquent, la prime unique diminue également,
— la prime annuelle suit la même tendance.

Ce comportement est conforme à la théorie actuarielle : une hausse du taux d'actualisation réduit la valeur actuelle des flux futurs.

7.3.3 Influence de l'âge et de la génération

L'âge x et la génération influencent directement les probabilités de survie :

— à âge élevé, les probabilités de survie diminuent, ce qui réduit la valeur attendue des prestations,
— pour une génération plus récente, la mortalité étant plus faible, la prime peut être plus élevée.

Ces effets sont intégrés naturellement dans les calculs via ${}_tp_x(\text{gen})$.

7.4 Relations entre variables (analyse conceptuelle)

Même sans représentation graphique, plusieurs relations structurelles peuvent être identifiées :

— **Linéarité en** A : la prime unique est proportionnelle à A.

— **Non-linéarité en** i : dépendance exponentielle via v^k.

— **Effet multiplicatif de la survie** : produit de probabilités ${}_tp_x$.

— **Structure par morceaux pour la réserve** : dépendance forte au régime temporel.

Ces caractéristiques expliquent pourquoi un modèle linéaire simple est insuffisant pour capturer l'ensemble des relations, en particulier pour la réserve.

7.5 Conclusion de l'étude descriptive

L'étude descriptive confirme que :

— le dataset est cohérent d'un point de vue actuariel,

— les ordres de grandeur sont plausibles,

— les variables explicatives couvrent correctement l'espace des paramètres,

— les relations entre variables sont fortement non-linéaires.

Ces observations justifient le choix d'un modèle non-linéaire (HistGradientBoosting) pour l'étape de Machine Learning et préparent l'analyse des performances présentée dans les chapitres suivants.

Chapitre 8

Machine Learning : modèles et protocole

8.1 Objectif ML

L'objectif du volet Machine Learning est d'approximer le moteur actuariel déterministe par une fonction statistique :

$$\hat{f} : (x, m, m', n, i, A, \text{generation}, t) \longmapsto (U, P, V_t),$$

où :

— U : prime unique nette,

— P : prime annuelle nivelée,

— V_t : réserve prospective au temps t.

Le moteur actuariel constitue ici la **vérité terrain**. Le ML n'a pas pour objectif de remplacer la théorie actuarielle, mais de montrer qu'un modèle appris peut approximer efficacement une fonction complexe composée :

— de produits de probabilités de survie,

— d'exponentielles d'actualisation,

— de divisions (cas de la prime annuelle),

— de structures par morceaux (cas de la réserve).

L'enjeu est donc double :

— mesurer la capacité d'approximation,

— analyser les limites selon la complexité de la target.

8.2 Features et targets

Les variables explicatives (features) sont directement les paramètres contractuels :

$$(x, m, m', n, i, A, \text{generation}, t).$$

On distingue :

— variables démographiques : x, `generation`,

— variables temporelles : m, m', n, t,

— variables financières : i, A.

Les variables cibles (targets) sont :

$$U, \quad P, \quad V_t.$$

Le problème est traité comme **trois régressions séparées**, une pour chaque target. Cette approche permet :

— d'évaluer la difficulté propre à chaque quantité,

— de comparer les performances selon la complexité actuarielle.

8.3 Préparation des données

8.3.1 Séparation Train/Test

Le dataset est divisé selon un schéma simple :

$$80\% \text{ données d'entraînement} \quad / \quad 20\% \text{ données de test.}$$

Le split est réalisé de manière aléatoire avec un **random seed fixé** afin d'assurer la reproductibilité des résultats.

8.3.2 Pourquoi un simple split ?

Étant donné que le dataset est généré aléatoirement et ne présente pas de dépendance temporelle réelle (les observations sont indépendantes), un simple split aléatoire est suffisant. L'objectif n'est pas de faire une optimisation fine des hyperparamètres mais d'analyser la capacité d'approximation globale.

8.4 Modèles testés

8.4.1 Baseline linéaire : Ridge Regression

Le modèle Ridge est une régression linéaire pénalisée :

$$\hat{\beta} = \arg\min_{\beta} \left(\|y - X\beta\|^2 + \lambda \|\beta\|^2 \right).$$

La pénalisation λ permet de stabiliser l'estimation en présence de corrélations entre features.

Rôle dans le projet. Ridge joue le rôle de **baseline**. Il permet de tester si une relation approximativement linéaire existe entre les paramètres et les targets.

Limites attendues. Or, les formules actuarielles comportent :

— des produits de probabilités,

— des puissances de v,

— des divisions,

— des changements de régime temporel.

On s'attend donc à ce que Ridge soit insuffisant, notamment pour la réserve.

8.4.2 Modèle non-linéaire : HistGradientBoostingRegressor (HGB)

Le modèle HistGradientBoostingRegressor est un modèle d'arbres boostés basé sur des partitions successives de l'espace des features.

Il présente plusieurs avantages :

— capture naturelle des **interactions non-linéaires**,

— bonne performance sur données tabulaires,

— robustesse aux échelles différentes des variables,

— pas besoin de standardisation préalable.

Pourquoi ce choix est cohérent ici ? Les relations actuarielles ont une structure hautement non-linéaire :

— dépendance exponentielle en i,

— dépendance multiplicative via ${}_tp_x$,

— dépendance rationnelle pour P,

— dépendance par morceaux pour V_t.

Un modèle basé sur des arbres est particulièrement adapté à ce type de structure.

8.5 Métriques d'évaluation

Les performances sont évaluées sur le jeu de test à l'aide de trois métriques complémentaires :

8.5.1 R^2 (coefficient de détermination)

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}.$$

Cette métrique mesure la proportion de variance expliquée par le modèle. Elle permet de comparer facilement la qualité d'approximation entre targets.

8.5.2 MAE (Mean Absolute Error)

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|.$$

La MAE mesure l'erreur moyenne absolue et donne une interprétation directe en unités monétaires.

8.5.3 RMSE (Root Mean Squared Error)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}.$$

La RMSE pénalise davantage les grandes erreurs que la MAE.

Pourquoi utiliser plusieurs métriques ?

— R^2 mesure la qualité globale d'explication.

— MAE et RMSE mesurent l'erreur en euros.

— La comparaison MAE/RMSE permet de détecter la présence d'erreurs extrêmes.

Il est important de souligner qu'un coefficient de détermination élevé ne constitue pas à lui seul une validation méthodologique suffisante.

L'analyse conjointe de R2, MAE et RMSE permet :

— d'évaluer la qualité globale d'ajustement, — d'interpréter les erreurs en unités monétaires, — de détecter la présence éventuelle d'erreurs extrêmes.

Cette approche multi-métrique garantit une analyse plus robuste des performances.

8.6 Learning Curves

Afin d'analyser l'effet de la taille du dataset, on calcule les performances pour différentes valeurs de N :

$$N \in \{50, 100, 200, 500, 1000\}.$$

Cette analyse permet de répondre à la question :

> Le modèle converge-t-il vers une approximation stable lorsque la taille du dataset augmente ?

On observe généralement :

— convergence rapide pour la prime unique,

— convergence plus lente pour la prime annuelle,

— convergence encore plus lente pour la réserve.

8.7 Feature Importance

Pour interpréter le modèle HGB, on utilise une **permutation importance** basée sur la baisse de R^2 lorsque l'on permute une feature.

Cette méthode permet d'identifier :

— les variables financières dominantes (A, i),

— les variables temporelles structurantes (m', n, t),

— les variables démographiques (x, `generation`).

Cette étape est importante pour valider que le modèle apprend une structure cohérente avec l'intuition actuarielle.

8.8 Synthèse méthodologique

Le protocole ML adopté est volontairement simple mais rigoureux :

— génération d'un dataset reproductible,

— séparation claire train/test,

— comparaison d'un modèle linéaire et d'un modèle non-linéaire,

— analyse multi-métriques,

— étude de la convergence via learning curves,

— interprétation via feature importance.

Cette approche permet d'analyser non seulement la performance brute, mais aussi la structure et la stabilité de l'approximation.

Chapitre 9

Résultats ML

9.1 Scores obtenus

Les résultats obtenus sur un dataset de taille $N \approx 1000$ sont :

Table 9.1 – Comparaison Ridge vs HGB (Test set).

Target	Modèle	MAE	RMSE	R^2
single_premium	Ridge	6474.85	9566.97	0.6619
single_premium	HGB	2098.07	4785.80	0.9154
annual_premium	Ridge	3865.06	9894.92	0.2154
annual_premium	HGB	1487.88	5757.88	0.7343
reserve	Ridge	4177.21	7145.22	0.4709
reserve	HGB	2688.48	5840.24	0.6465

9.2 Commentaires

— La **prime unique** est très bien approximée par HGB ($R^2 \approx 0.92$). Cela confirme que la fonction actuarielle, bien que déterministe, est suffisamment régulière pour être apprise par un modèle non-linéaire.

— La **prime annuelle** est plus difficile : elle implique une division par une annuité-due $\ddot{a}_{x:\overline{m}}$. Un modèle linéaire échoue largement, tandis que HGB atteint un niveau correct ($R^2 \approx 0.73$).

— La **réserve** est la plus complexe : dépendance forte en t, changement de régime avant/après m', et structure de type « montée puis descente ». Le R^2 reste plus faible (≈ 0.65) mais reste exploitable.

La réserve dépend simultanément :

— du temps t, — de la structure contractuelle (m, m', n), — et d'un changement de régime avant/après paiement des primes.

Cette structure par morceaux introduit une discontinuité locale dans la fonction cible, ce qui rend son approximation plus difficile pour un modèle statistique.

Ainsi, la difficulté d'apprentissage reflète directement la complexité mathématique de la formule actuarielle.

Chapitre 10

Learning curves : évolution des performances en fonction de N

L'énoncé demande l'étude de l'évolution des performances pour différentes tailles de dataset. On calcule R^2 sur le test set pour des tailles typiques :

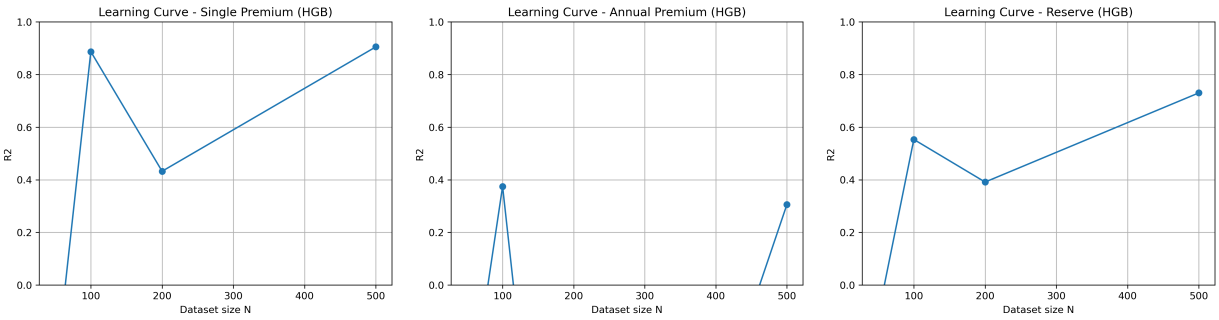
$$N \in \{50, 100, 200, 500, 1000\}.$$

10.1 Résultats attendus

On observe généralement :

— convergence rapide pour la prime unique (structure plus simple),

— convergence plus lente pour la prime annuelle et surtout pour la réserve.



(a) Single premium (b) Annual premium (c) Reserve

FIGURE 10.1 – Learning curves (HGB) : évolution de R^2 en fonction de N.

10.2 Interprétation des learning curves

La Figure 10.1 illustre l'évolution du coefficient de détermination R^2 en fonction de la taille du dataset N pour le modèle HistGradientBoosting.

On observe plusieurs phénomènes structurants.

Prime unique

La prime unique présente une convergence rapide lorsque N augmente. Dès quelques centaines d'observations, le modèle atteint un R^2 élevé.

Cela s'explique par la structure relativement simple de la formule :

$$U = A \cdot \sum v^{k+1}{}_k p_x.$$

La dépendance est essentiellement multiplicative et monotone en A et i, ce qui est bien capté par un modèle d'arbres boostés.

Prime annuelle

La convergence est plus lente pour la prime annuelle. Le R^2 progresse avec N, mais reste inférieur à celui de la prime unique.

Cette différence provient du fait que :

$$P = \frac{U}{\ddot{a}_{x:\overline{m}}},$$

introduisant une division par une quantité elle-même non linéaire. La structure rationnelle rend l'approximation plus complexe.

Réserve

La réserve constitue la target la plus difficile. Sa structure dépend fortement du temps t et comporte des changements de régime :

— phase avant différé,
— phase de paiement des primes,
— phase de service de la rente,
— extinction progressive.

Cette dynamique par morceaux explique :

— une variance plus importante,
— une convergence plus lente,
— un R^2 structurellement plus faible.

Conclusion

Les learning curves confirment que :

— le modèle HGB apprend efficacement la structure actuarielle,
— la performance augmente avec la taille du dataset,
— la complexité mathématique croissante des targets se traduit directement dans la difficulté d'approximation.

Cela valide la cohérence entre la théorie actuarielle et les résultats empiriques du Machine Learning.

Chapitre 11

Feature importance

Pour interpréter le modèle HGB, on utilise une **permutation importance** (baisse de R^2 lorsque l'on permute une feature).

11.1 Intuition

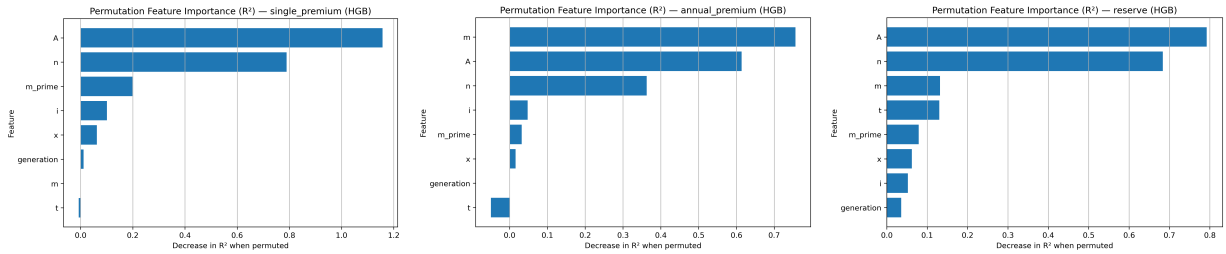
On s'attend à :

— A et i fortement importants (montant et actualisation),

— m' et n importants (différé et durée de service),

— x et `generation` importants (mortalité),

— t très important pour la réserve.



(a) Single premium (b) Annual premium (c) Reserve

FIGURE 11.1 – Permutation feature importance (HGB).

Chapitre 12

Application Streamlit

12.1 Démarche de développement

Le développement du projet s'est déroulé en deux étapes distinctes.

Phase 1 : Notebook exploratoire

Dans un premier temps, l'ensemble du travail a été réalisé dans un notebook Python :

- implémentation complète du moteur actuariel,
- génération et étude descriptive du dataset,
- entraînement des modèles de Machine Learning,
- analyse des performances (MAE, RMSE, R^2),
- construction des learning curves,
- étude des feature importances.

Le notebook a permis :

- de vérifier la cohérence mathématique des formules,
- d'identifier les comportements numériques atypiques,
- d'ajuster progressivement le pipeline ML.

Il constitue ainsi la base analytique et scientifique du projet.

Phase 2 : Application Streamlit

Dans un second temps, le moteur actuariel et les modèles entraînés ont été intégrés dans une application interactive développée avec `Streamlit`.

L'objectif était double :

- proposer une interface utilisateur claire,
- démontrer concrètement la capacité d'approximation du ML.

12.2 Architecture de l'application

L'application est structurée en trois pages principales :

- **Moteur Actuariat**
  - saisie des paramètres $(x, m, m', n, i, A, \text{generation}, t)$,
  - calcul exact des quantités (U, P, V_t),
  - affichage numérique et courbe des réserves.
- **Moteur ML**

  - prédiction des mêmes quantités via le modèle HGB,
  - comparaison directe avec le moteur actuariel,
  - affichage des écarts absolus et relatifs.
- **Performance & Training**
  - génération et sauvegarde du dataset,
  - entraînement des modèles,
  - affichage des métriques,
  - learning curves,
  - feature importance.

12.3 Choix de conception

La séparation en plusieurs pages présente plusieurs avantages :

- éviter le ré-entraînement systématique des modèles,
- séparer clairement calcul exact et approximation ML,
- améliorer la lisibilité pédagogique,
- permettre une démonstration progressive lors d'une soutenance.

Le moteur actuariel joue le rôle de référence théorique, tandis que le moteur ML illustre la capacité d'approximation statistique.

12.4 Apport pédagogique

L'application Streamlit transforme un projet académique en outil interactif :

- visualisation immédiate de l'impact d'un paramètre,
- comparaison en temps réel Actuariat vs ML,
- compréhension intuitive des écarts et des limites.

Cette approche renforce la dimension appliquée du projet et met en valeur l'intégration entre science actuarielle et intelligence artificielle.

Chapitre 13

Conclusion

Ce projet montre qu'un moteur actuariel basé sur une table générationnelle (TGF05) peut être implémenté de manière robuste et utilisé pour générer un dataset synthétique conforme à l'énoncé. Sur ce dataset, un modèle non-linéaire de type HistGradientBoosting approxime efficacement les quantités actuarielles :

— très bonne performance sur la prime unique ($R^2 \approx 0.92$),

— bonne performance sur la prime annuelle ($R^2 \approx 0.73$),

— performance correcte sur la réserve ($R^2 \approx 0.65$), plus difficile à prédire du fait de la dynamique temporelle et des changements de régime.

Les learning curves confirment que l'augmentation de N améliore la performance, en particulier pour les targets les plus complexes. Enfin, la permutation importance permet d'interpréter qualitativement les facteurs les plus influents (montant A, taux i, paramètres de durée, mortalité via x et la génération, et temps t pour la réserve).

Perspectives. Une amélioration possible consiste à enrichir les features (feature engineering) pour mieux capturer le régime avant/après m' (ex. $m' - t$ ou un indicateur de début de service), afin d'améliorer la prédiction de la réserve.